

LAB 5 REPORT

Interfacing LCD with I2C

Name: Michael McCorkell

Student ID: N01500049

Course: Microcontrollers – Winter 2026

Instructor: Nilufar Poshtiban

Date: April 3, 2026

Abstract

This lab focused on getting a 16x2 LCD that could interface with the TM4C123 microcontroller via I2C via the PCF8574 I/O expander on the LCD module's rear. Three objectives were completed: displaying a basic message, generating a 1-second counter on the screen, and finally changing the entire system from I2C0 to I2C1, which required both wiring and programming changes. The 10 lab questions on I2C theory, pull-up resistors, bus conflicts, and portability to other platforms are all answered.

Introduction

I2C (Inter-Integrated Circuit) is a two-wire serial protocol that enables a microcontroller to connect with a wide range of peripheral devices using a single data line (SDA) and a clock line (SCL). The TM4C123 serves as the master, with the LCD module acting as the slave. The LCD is a conventional HD44780-based 16x2 display; however, because the controller uses a parallel interface, a PCF8574 chip is mounted to the back to convert I2C data into the parallel signals required by the LCD.

Because both SDA and SCL are open-drain lines, pull-up resistors are necessary to keep them HIGH while nothing is driving them downward. The PCF8574's slave address is often 0x27 or 0x3F, depending on how the module's address pins are configured. In this laboratory, 0x27 was used.

Objectives

- Wire the LCD module to the TM4C123 using I2C0 on Port B (PB2 = SCL, PB3 = SDA).
- Display text messages on the LCD, including at specific row and column positions.
- Build an incrementing counter that updates on the LCD every second.
- Migrate the design from I2C0 to I2C1 and document all the changes required.
- Answer the theory questions on I2C, pull-up resistors, bus conflicts, and porting.

Equipment

- TI TM4C123GH6PM Tiva C LaunchPad
- LCD 1602 with PCF8574 I2C backpack
- 10 k Ω potentiometer (contrast adjustment)
- Breadboard and connecting wires
- Computer with Keil MDK IDE

Activity 1 – Basic LCD Display

Circuit Setup

The LCD module was wired up according to the pin configuration from the lab sheet. The two I2C lines connect to Port B on the microcontroller:

- PB2 → SCL
- PB3 → SDA
- LCD VCC → 5V
- LCD GND → GND

The potentiometer on the back of the LCD module was adjusted until text was clearly visible. Without that adjustment the screen either shows nothing or solid black blocks.

Procedure

The LCD_I2C_Template source file was loaded into Keil, compiled, and flashed to the board. After power-on the LCD displayed "Hello Tiva" on the first line as expected. The second part of the activity was to move that same message to line 2, column 5. This involved calling the cursor positioning function with row = 1 and column = 4 (zero-indexed) before writing the string.

Relevant Code

```
LCD_setCursor(0, 0);          // row 0, col 0
LCD_writeString("Hello Tiva");

// Part 2 - line 2, column 5
LCD_setCursor(1, 4);          // row 1, col 4 (0-indexed)
LCD_writeString("Hello Tiva");
```

Result

First run showed "Hello Tiva" on the top left. After modifying the cursor position it appeared on the second line starting at column 5. The potentiometer needed a small tweak to get the contrast right on the second row.

Activity 2 – Incrementing Counter

Overview

This activity is based on Activity 1. The goal was to center "Counter:" on line 1 and then show a number below it on line 2 that increased every second. To appear balanced when centered on a 16-character display, the text must begin in the appropriate column. "Counter:" is 8 characters long, thus it should begin in column 4 and be centered.

Implementation

Within a loop, a counter variable was incremented, and the delayMs function from the previous lab was used to apply a 1-second delay. Each repeat, the second line is cleaned before recording the counter value, guaranteeing that previous digits do not remain as the number increases (for example, from 9 to 10).

Relevant Code

```
LCD_setCursor(0, 4);
LCD_writeString("Counter:");

int count = 0;
while(1){
    LCD_setCursor(1, 0);
    LCD_writeString("          "); // clear line 2
    LCD_setCursor(1, 7);           // center the number
    LCD_writeInt(count);
    count++;
    delayMs(1000);
}
```

Result

"Counter:" displayed centered on line 1. The number on line 2 started at 0 and incremented every second. Clearing the line before writing each new value was important because without it the old digits would stay on screen when the number got longer.

Activity 3 – Switching from I2C0 to I2C1

Overview

This activity required migrating the LCD from I2C0 (PB2/PB3) to I2C1, which uses a different set of pins on the TM4C123. Several things had to change in both the hardware wiring and the code.

Pin Connection Changes

I2C1 on the TM4C123 is mapped to Port A, pins PA6 and PA7:

- PA6 → SCL (I2C1)
- PA7 → SDA (I2C1)

The wires on the breadboard were moved from PB2/PB3 to PA6/PA7 accordingly.

Peripheral Initialization Changes

The clock enable register had to be updated. Instead of enabling I2C0 and Port B, I2C1 and Port A are enabled:

```
// Old (I2C0):
SYSCTL->RCGCI2C |= 0x01;    // enable I2C0
SYSCTL->RCGCGPIO |= 0x02;    // enable Port B

// New (I2C1):
SYSCTL->RCGCI2C |= 0x02;    // enable I2C1
SYSCTL->RCGCGPIO |= 0x01;    // enable Port A
```

GPIO Configuration Changes

Port A pins PA6 and PA7 need to be set up for I2C alternate function. The AFSEL, ODR, DEN, and PCTL registers all need to point to Port A instead of Port B, and the PCTL value changes to select the I2C alternate function for those specific pins.

```
// Old (Port B, I2C0):
GPIOB->AFSEL |= 0x0C;        // PB2, PB3
GPIOB->ODR    |= 0x0C;
GPIOB->DEN    |= 0x0C;
GPIOB->PCTL   |= 0x3300;      // AF3 for I2C0

// New (Port A, I2C1):
GPIOA->AFSEL |= 0xC0;         // PA6, PA7
GPIOA->ODR    |= 0xC0;
GPIOA->DEN    |= 0xC0;
GPIOA->PCTL   |= 0x33000000;  // AF3 for I2C1
```

I2C Base/Module Changes

Every reference to I2C0 in the code was changed to I2C1. This includes the master control register, the slave address register, the data register, and the status register. Wherever the code used the I2C0 base address it now uses I2C1.

```
// Old:  
I2C0->MSA = (LCD_ADDR << 1);  
I2C0->MCS = ...;  
  
// New:  
I2C1->MSA = (LCD_ADDR << 1);  
I2C1->MCS = ...;
```

Result

After moving the wires and updating the code, the LCD displayed the custom message over I2C1 without issues. The initialization sequence and timing were the same as before since it's still the same PCF8574 and HD44780, just on a different I2C module.

Picture of circuit:



Results Summary

Activity	Outcome	Key Observation
Activity 1 – Hello Tiva	Success	Text displayed at default position and then repositioned to row 2, col 5 using cursor function.
Activity 2 – Counter	Success	"Counter:" centered on line 1; incrementing number on line 2 updated every 1 second.
Activity 3 – I2C1 Migration	Success	Wires moved to PA6/PA7, clock and GPIO registers updated, I2C0 base replaced with I2C1 throughout.

Discussion

Activity 1 was mostly focused with getting familiar with the I2C LCD library. The cursor positioning function is zero-indexed, therefore column 5 on the lab sheet equates to passing 4 in code, a little but crucial distinction. Adjusting the potentiometer before doing anything else saved a lot of confusion because the LCD appears completely blank when the contrast is turned off. Activity 2 involved basic counter logic. The only item that needed attention was clearing the line before recording the following number every second. Going from a single digit to two digits works. It's fine to just write the number without first clearing it, however changing from 99 to 100 will leave a ghost character from the previous value. Activity three was the most involved. Switching I2C modules necessitates more than a single line change; the clock enabling bits, the GPIO port, the alternate function codes in PCTL, and all register accesses in the I2C driver routines must be altered. It was a great exercise in understanding how the peripheral base addresses and register mappings work on this microcontroller. When all of the changes were made, it worked the first time, suggesting that the adjustments were correct.

Conclusion

All three drills went as anticipated. I2C proved to be a rather clean way to operate the LCD because it just takes two wires regardless of how many devices are on the bus. The PCF8574 does all of the hard lifting in converting serial data back into the parallel signals required by the HD44780, thus the software side is mostly concerned with transferring the proper bytes at the appropriate times. Activity 3 illustrated that switching I2C modules mostly involves altering pin assignments and peripheral register references throughout the driver code.

Lab Questions

Q1. Why are pull-up resistors required in I2C instead of pull-down resistors?

I2C uses open-drain lines, which implies that devices can only pull the line LOW and cannot actively drive it HIGH. When nothing is pulling the line down, the pull-up resistors restore it to a HIGH state. If you utilized pull-downs instead, the line would remain permanently LOW, making communication impossible because no device can push it HIGH on its own.

Q2. What would happen if SDA is held LOW by a faulty device?

This is referred to as bus lockdown. If anything is caught holding SDA down, the master sees the bus as constantly busy and is unable to supply a START condition because a START requires SDA to move from HIGH to DOWN while SCL is HIGH. All other devices on the bus are likewise prohibited. The standard treatment is to power cycle the damaged device or have the master clock out dummy pulses on SCL in an attempt to free it, but there is no proven software-only approach.

Q3. Compare I2C with SPI:

Hardware complexity: I2C requires only two wires (SDA and SCL) regardless of the number of devices added, making wiring easy. SPI requires at least four lines (MOSI, MISO, SCLK, and CS), as well as a separate chip-select for each additional device, which can get messy quickly. Speed: SPI is faster. Standard I2C has a maximum frequency of 100 kHz, or 400 kHz in fast mode. SPI can operate at tens of MHz depending on the hardware. It makes no difference to an LCD, but it does for quick sensors or memory.

Scalability: I2C scales better in terms of wiring because you can simply add more devices to the same two lines as long as each has a different address. SPI requires a new CS pin for each device, which burns through GPIO pins quickly.

Q4. How does the master detect whether a slave device is present on the bus?

The master delivers the START condition, followed by the 7-bit slave address with the write bit set. When a device with that address is on the bus, it pulls SDA low during the ACK bit slot. If no response is received, SDA remains HIGH (NACK), indicating to the master that nothing is present. This is exactly how I2C scanner tools work: they iterate through all 128 potential addresses, checking for an ACK at each.

Q5. What is the address of the I2C LCD?

The PCF8574 I/O expander on the LCD module has a 7-bit address set by the board's A0, A1, and A2 solder pads. The most common address is 0x27 (all address pins are low), while other modules utilize 0x3F. In this laboratory, the address was 0x27.

Q6. Can multiple I2C devices be connected on the same bus?

Yes, that is one of the primary benefits of I2C. As long as each device has a unique address, they can all use the same SDA and SCL lines. As a result, a temperature sensor, an LCD, and an EEPROM may all operate simultaneously on two wires without the need for additional gear.

Q7. If two devices have the same I2C address, how can the conflict be resolved?

Several choices based on the hardware. Some devices contain address pins (such as A0/A1/A2 on the PCF8574) that can be tied high or low to change the address, allowing one device to be reconfigured at a different address. If neither device supports address changes, you have two options: put one device on a different I2C bus (e.g., I2C1 versus I2C0) or use an I2C multiplexer chip, such as the TCA9548A, which routes traffic to one bus at time.

Q8. How does the LCD differentiate between command and data?

This is controlled via the RS pin (Register Select). When RS is LOW, the LCD interprets the incoming byte as a command, such as clearing the screen, setting the cursor location, or configuring display mode. When RS is HIGH, the LCD interprets it as character data to be shown. The PCF8574 incorporates the RS bit, as well as the data nibble and enable pulse, into the byte delivered over I2C.

Q9. Why is a delay required during LCD initialization?

After applying voltage to the HD44780 controller, it takes some time to power up and stabilize. The datasheet specifies a minimum delay of 15 ms after power-on before sending any commands, followed by lower delays between early initialization phases. If the delays are missed, the LCD may miss the initial few commands and enter an undefined state in which nothing appears on screen at all.

Q10. What changes are required to port this code to STM32?

Several things would require updating. The peripheral clock enables the usage of Tiva-specific SYSCTL registers, which would be replaced by STM32 RCC calls. The GPIO setup (AFSEL, DEN, PCTL, and ODR) corresponds to STM32 GPIO registers with various names and alternate function numbers. The I2C register names and initialization sequences are also different: Tiva uses MSA/MCS/MDR, whereas STM32 uses CR1/CR2/DR or their HAL equivalents. The actual LCD logic for transferring nibbles, toggling the enable pin, and formatting the RS/EN/backlight bits remains unchanged because it is based on the HD44780 and PCF8574 protocols rather than the microcontroller.

References

Mazidi, M. A., Chen, S., Naimi, S., & Naimi, S. (2017). TI ARM Peripherals Programming and Interfacing: Programming ARM Cortex-M4 TM4C123G with C. MicroDigitalEd.com.

Texas Instruments. (2014). Tiva TM4C123GH6PM Microcontroller Data Sheet (Rev. E).
<https://www.ti.com/lit/ds/spms376e/spms376e.pdf>

NXP Semiconductors. PCF8574 Remote 8-bit I/O Expander for I2C Bus Datasheet.
<https://www.nxp.com/docs/en/data-sheet/PCF8574.pdf>

Hitachi. HD44780U LCD Controller Datasheet.
<https://www.sparkfun.com/datasheets/LCD/HD44780.pdf>

Writing Advice Home. University of Toronto.
<https://advice.writing.utoronto.ca/types-of-writing/lab-report/>